

Angular JS

Unit III

Topics

- Angular JS: Introduction
- MVC Architecture
- Directives , Expression , Controllers , Filters , Tables,
- HTML DOM ,
- Modules ,
- Forms ,
- AJAX ,
- Scope and Services

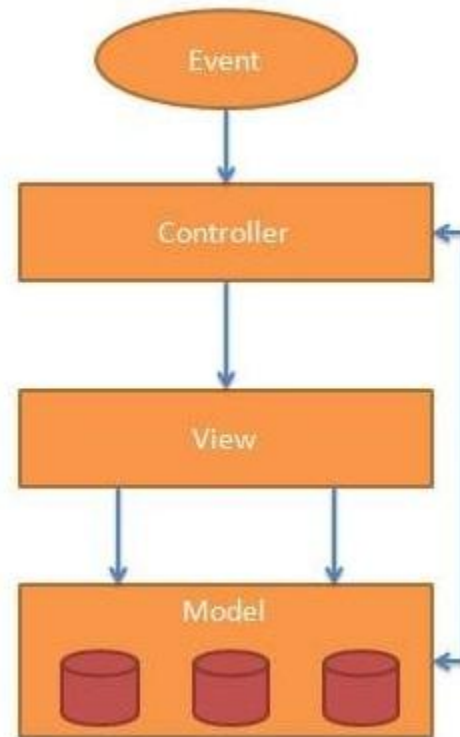
Introduction

- Angular JS is an open source JavaScript framework by Google to build web applications. It can be freely used, changed and shared by anyone.
- **Two way data binding:** AngularJS creates a two way data-binding between the select element and the orderProp model. orderProp is then used as the input for the orderBy filter.
- **Model View Controller:** In Angular JS, it is very easy to develop application in a clean MVC way. You just have to split your application code into MVC components i.e. Model, View and the Controller.

AngularJS MVC Architecture

- MVC stands for Model View Controller. It is a software design pattern for developing web applications. It is very popular because it isolates the application logic from the user interface layer and supports separation of concerns.

MVC Architecture



MVC Architecture

1. **Model:** It is responsible for managing application data. It responds to the requests from view and to the instructions from controller to update itself.
2. **View:** It is responsible for displaying all data or only a portion of data to the users. It also specifies the data in a particular format triggered by the controller's decision to present the data. They are script-based template systems such as JSP, ASP, PHP and very easy to integrate with AJAX technology.
3. **Controller:** It is responsible to control the relation between models and views. It responds to user input and performs interactions on the data model objects. The controller receives input, validates it, and then performs business operations that modify the state of the data model.

- include the AngularJS JavaScript file in the HTML page

```
<!DOCTYPE html> <html>

<head>

  <script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.2
.5/angular.min.js"></script>

</head>

<body>

. . .

</body>

</html>
```

First Program

- ```
<!DOCTYPE html>
<html lang="en-US">
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.
9/angular.min.js"></script>
<body>

 <div ng-app="">
 <p>Name : <input type="text" ng-model="name"></p>
 <h1>Hello {{name}}</h1>
 </div>

</body>
</html>
```

## Example 2

```
<body>
 <p>Change the value of the input field:</p>
 <div ng-app="" ng-init="myCol='yellow'">
 <input style="background-color:{{myCol}}"
ng-model="myCol">
 </div>
</body>
```

yellow

green

skyblue

# AngularJS Expressions

AngularJS expressions can be written inside double braces: `{{ expression }}`.

AngularJS expressions can also be written inside a directive: `ng-bind="expression".`

AngularJS will resolve the expression, and return the result exactly where the expression is written.

- **AngularJS expressions** are much like **JavaScript expressions**:
- They can contain literals, operators, and variables.
- Example `{{ 5 + 5 }}` or `{{ firstName + " " + lastName }}`

# AngularJS Expressions vs. JavaScript Expressions

- Like JavaScript expressions, AngularJS expressions can contain literals, operators, and variables.
- Unlike JavaScript expressions, AngularJS expressions can be written inside HTML.
- AngularJS expressions do not support conditionals, loops, and exceptions, while JavaScript expressions do.
- AngularJS expressions support filters, while JavaScript expressions do not.

# AngularJS Numbers

- AngularJS numbers are like JavaScript numbers:
- Example

- `<div ng-app="" ng-init="quantity=1;cost=5">`

`<p>Total in dollar: {{ quantity * cost }}</p>`

`</div>`

# AngularJS Strings

- AngularJS strings are like JavaScript strings:
- Example
- `<div ng-app="" ng-init="firstName='John';lastName='Doe'">`  
`<p>The name is {{ firstName + " " + lastName }}</p>`  
`</div>`

Output:

The name is John Doe

# Objects

- AngularJS objects are like JavaScript objects:
- Example
- `<div ng-app="" ng-init="person={firstName: 'John', lastName: 'Doe'}">`  
`<p>The name is {{ person.lastName }}</p>`  
`</div>`

Output:  
The name is Doe

# Arrays

- AngularJS arrays are like JavaScript arrays:
- Example
- `<div ng-app="" ng-init="points=[1,15,19,2,40]">`  
`<p>The result is {{ points[2] }}</p>`  
`</div>`

Output:  
The result is 19

# AngularJS Directives

- AngularJS lets you extend HTML with new attributes called **Directives**.
- AngularJS has a set of built-in directives which offers functionality to your applications.
- AngularJS also lets you define your own directives.

AngularJS directives are extended HTML attributes with the prefix **ng-**.

The **ng-app** directive initializes an AngularJS application.

The **ng-init** directive initializes application data.

The **ng-model** directive binds the value of HTML controls (input, select, textarea) to application data.

# Example

```
<!DOCTYPE html> <html>

<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6
.9/angular.min.js"></script>

<body>

<div ng-app="" ng-init="firstName='John'">
<p>Input something in the input box:</p>
<p>Name: <input type="text" ng-model="firstName"></p>
<p>You wrote: {{ firstName }}</p>
</div> </body> </html>
```

# output

---

Input something in the input box:

Name:

You wrote: John

## Example 2

```
<div ng-app="" ng-init="quantity=1;price=5">
```

```
Quantity: <input type="number" ng-model="quantity">
```

```
Costs: <input type="number" ng-model="price">
```

```
Total in dollar: {{ quantity * price }}
</div>
```

### Cost Calculator

Quantity:  Price:

Total in dollar: 5

## Repeating HTML Elements

The **ng-repeat** directive repeats an HTML element:

```
<div ng-app="" ng-init="names=['Jani', 'Hege', 'Kai']">

 <li ng-repeat="x in names">
 {{ x }}

</div>
```

Looping with ng-repeat:

- Jani
- Hege
- Kai

## Repeating HTML Elements

```
<div ng-app="" ng-init="names=[
 {name: 'Jani', country: 'Norway'},
 {name: 'Hege', country: 'Sweden'},
 {name: 'Kai', country: 'Denmark'}]">

 <li ng-repeat="x in names">
 {{ x.name + ', ' + x.country }}

</div>
```

### Looping with objects:

- Jani, Norway
- Hege, Sweden
- Kai, Denmark

## The ng-app Directive

The **ng-app** directive defines the root element of an AngularJS application.

The **ng-app** directive will auto-bootstrap (automatically initialize) the application when a web page is loaded.

## The ng-init Directive

The **ng-init** directive defines initial values for an AngularJS application.

## The ng-model Directive

The `ng-model` directive binds the value of HTML controls (input, select, textarea) to application data.

The `ng-model` directive can also:

- Provide type validation for application data (number, email, required).
- Provide status for application data (invalid, error).
- Provide CSS classes for HTML elements.
- Bind HTML elements to HTML forms.

# Create New Directives

In addition to all the built-in AngularJS directives, you can create your own directives.

New directives are created by using the `.directive` function.

To invoke the new directive, make an HTML element with the same tag name as the new directive.

When naming a directive, you must use a camel case name, `myTestDirective`, but when invoking it, you must use - separated name, `my-test-directive`:

# Example

```
<body ng-app="myApp">
```

```
<my-test> </my-test>
```

```
<script>
```

```
var app = angular.module("myApp", []);
```

```
app.directive("myTest", function() {
```

```
 return {
```

```
 template : "<h1>Made by me!</h1>"
```

```
 };
```

```
});</script>
```

```
</body>
```

```
<body ng-app="myApp">
```

```
<!-- Custom directive 'mytest' will be replaced by its
template when rendered
```

```
<mytest></mytest>
```

```
<script>
```

```
 var app = angular.module("myApp", []); // no dependency
```

```
 // Create a custom directive named 'mytest'
```

```
 app.directive("mytest", function() {
```

```
 return {
```

```
 // The HTML template that will replace <mytest></mytest>
```

```
 template : "<h1>Made by me!</h1>"
```

```
 }; }); </script> </body> </html>
```

You can invoke a directive by using:

- Element name :

```
<my-test-directive></my-test-directive>
```

- Attribute :

```
<div my-test-directive></div>
```

- Class :

```
<div class="my-test-directive"></div>
```

- Comment :

```
<!-- directive: my-test-directive -->
```

# Restrictions

By adding a `restrict` property with the value `"A"`, the directive can only be invoked by `attributes`:

```
var app = angular.module("myApp", []);
app.directive("myTestDirective", function() {
 return {
 restrict : "A",
 template : "<h1>Made by a directive!</h1>"
 };
});
```

# Restrictions

The legal restrict values are:

- **E** for Element name
- **A** for Attribute
- **C** for Class
- **M** for Comment

By default the value is **EA**, meaning that both Element names and attribute names can invoke the directive.

# AngularJS Controllers

AngularJS controllers control the data of AngularJS applications.

AngularJS controllers are regular JavaScript Objects.

The **ng-controller** directive defines the application controller.

A controller is a **JavaScript Object**, created by a standard JavaScript object constructor.

# Example

```
<script>
```

```
 var app = angular.module('myApp', []);
 app.controller('myCtrl', function($scope) {
 $scope.firstName = "John";
 $scope.lastName = "Doe";
 });
```

```
</script>
```

```
<div ng-app="myApp" ng-controller="myCtrl">
First Name: <input type="text" ng-model="firstName">

Last Name: <input type="text" ng-model="lastName">

Full Name: {{firstName + " " + lastName}}
</div>
```

Example:

[https://www.w3schools.com/angular/tryit.asp?filename=try\\_ng\\_c\\_ontroller](https://www.w3schools.com/angular/tryit.asp?filename=try_ng_c_ontroller)

# Controller Methods

```
<script>
var app = angular.module('myApp', []);
app.controller('personCtrl', function($scope) {
 $scope.firstName = "John";
 $scope.lastName = "Doe";
 $scope.fullName = function() {
 return $scope.firstName + " " + $scope.lastName;
 };
});
</script>
```

# Controller Methods

```
<div ng-app="myApp" ng-controller="personCtrl">
```

```
First Name: <input type="text" ng-model="firstName">

```

```
Last Name: <input type="text" ng-model="lastName">

```

```


```

```
Full Name: {{fullName()}}
```

```
</div>
```

## Controllers In External Files

```
<div ng-app="myApp" ng-controller="personCtrl">
First Name: <input type="text" ng-model="firstName">

Last Name: <input type="text" ng-model="lastName">

Full Name: {{fullName()}}

</div>
<script src="personController.js"></script>
```

# Filters

AngularJS provides filters to transform data:

- **currency** Format a number to a currency format.
- **date** Format a date to a specified format.
- **filter** Select a subset of items from an array.
- **limitTo** Limits an array/string, into a specified number of elements/characters.
- **lowercase** Format a string to lowercase.
- **number** Format a number to a string.
- **orderBy** Orders an array by an expression.
- **uppercase** Format a string to uppercase.

# Examples

Filters can be added to expressions by using the pipe character `|`, followed by a filter.

The `uppercase` filter format strings to uppercase:

```
<div ng-app="myApp" ng-controller="personCtrl">
<p>The name is {{ lastName | uppercase }}</p>
</div>
```

Output:  
The name is DOE

## Adding Filters to Directives

Filters are added to directives, like `ng-repeat`, by using the pipe character `|`, followed by a filter:

```
<div ng-app="myApp" ng-controller="namesCtrl">

 <li ng-repeat="x in names | orderBy:'country'">
 // <li ng-repeat="x in names | orderBy:'-country'">
 {{ x.name + ', ' + x.country }}

</div>
```

Looping with objects:

- Joe, Denmark
- Margareth, England
- Jani, Norway
- Hege, Norway
- Carl, Sweden
- Gustav, Sweden

# The currency Filter

```
<div ng-app="myApp" ng-controller="costCtrl">
```

```
<h1>Price: {{ price | currency:'₹' }}</h1>
```

**Price: ₹58.00**

```
//<h1>Price: {{ price | currency }}</h1>
```

**Price: \$58.00**

```
</div>
```

# scope

When you make a controller in AngularJS, you pass the `$scope` object as an argument:

When adding properties to the `$scope` object in the controller, the view (HTML) gets access to these properties.

In the view, you do not use the prefix `$scope`, you just refer to a property name, like `{{carname}}`.

# Scope

If we consider an AngularJS application to consist of:

- View, which is the HTML.
- Model, which is the data available for the current view.
- Controller, which is the JavaScript function that makes/changes/removes/controls the data.

Then the **scope is the Model**.

The scope is a JavaScript object with properties and methods, which are available for both the view and the controller.

# Example

```
<div ng-app="myApp" ng-controller="myCtrl">
<h1>{{carname}}</h1>
</div>
```

```
<script>
```

```
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
 $scope.carname = "Volvo";
});
```

```
</script>
```

[https://www.w3schools.com/angular/tryit.asp?filename=try\\_ng\\_scope\\_rootscope](https://www.w3schools.com/angular/tryit.asp?filename=try_ng_scope_rootscope)

# Root Scope

All applications have a `$rootScope` which is the scope created on the HTML element that contains the `ng-app` directive.

The `rootScope` is available in the entire application.

If a variable has the same name in both the current scope and in the `rootScope`, the application uses the one in the current scope.

```
<body ng-app="myApp">
 <p>The rootScope's favorite color:</p>
 <h1>{{color}}</h1>
 <div ng-controller="myCtrl">
 <p>The scope of the controller's favorite color:</p>
 <h1>{{color}}</h1>
 </div>
 <p>The rootScope's favorite color is still:</p>
 <h1>{{color}}</h1>
```

```
<script>
var app = angular.module('myApp', []);
app.run(function($rootScope) {
 $rootScope.color = 'blue';
});
app.controller('myCtrl', function($scope) {
 $scope.color = "red";
});
</script>
</body>
```

Activate Windows

Go to Settings to activate Windows.

# Forms

Forms in AngularJS provides data-binding and validation of input controls.

## Input Controls

Input controls are the HTML input elements:

- input elements
- select elements
- button elements
- textarea elements

## Data-Binding

Input controls provides data-binding by using the `ng-model` directive.

```
<input type="text" ng-model="firstname">
```

```
<script>
```

```
var app = angular.module('myApp', []);
```

```
app.controller('formCtrl', function($scope) {
```

```
 $scope.firstname = "John";
```

```
});
```

```
</script>
```

## Checkbox

A checkbox has the value `true` or `false`. Apply the `ng-model` directive to a checkbox, and use its value in your application.

Show the header if the checkbox is checked:

```
<form>
```

Check to show a header:

```
<input type="checkbox" ng-model="myVar">
```

```
</form>
```

```
<h1 ng-show="myVar">My Header</h1>
```

[https://www.w3schools.com/angular/tryit.asp?filename=try\\_ng\\_form\\_checkbox](https://www.w3schools.com/angular/tryit.asp?filename=try_ng_form_checkbox)

```
<form>
```

Select a topic:

```
<select ng-model="myVar">
```

```
<option value="">
```

```
<option value="dogs">Dogs
```

```
<option value="tuts">Tutorials
```

```
<option value="cars">Cars
```

```
</select>
```

```
</form>
```

## Radio Buttons

Bind radio buttons to your application with the `ng-model` directive.

Radio buttons with the same `ng-model` can have different values, but only the selected one will be used.

Display some text, based on the value of the selected radio button:

```
<form>
```

Pick a topic:

```
<input type="radio" ng-model="myVar" value="dogs">Dogs
```

```
<input type="radio" ng-model="myVar" value="tuts">Tutorials
```

```
<input type="radio" ng-model="myVar" value="cars">Cars
```

```
</form>
```

[https://www.w3schools.com/angular/tryit.asp?filename=try\\_ng\\_form\\_radio](https://www.w3schools.com/angular/tryit.asp?filename=try_ng_form_radio)

```
<div ng-switch="myVar">
 <div ng-switch-when="dogs">
 <h1>Dogs</h1>
 <p>Welcome to a world of dogs.</p>
 </div>
 <div ng-switch-when="tuts">
 <h1>Tutorials</h1>
 <p>Learn from examples.</p>
 </div>
 <div ng-switch-when="cars">
 <h1>Cars</h1>
 <p>Read about cars.</p>
 </div> </div>
```

# Example

```
<!DOCTYPE html>
<html ng-app="myApp">
<head>
 <title>AngularJS Form Example</title>
 <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="myCtrl">

 <!-- Form starts -->
 <form name="myform" ng-submit="submitForm()">
 <!-- Name input field -->
 <label for="name">Name:</label>
 <input type="text" id="name" ng-model="user.name" required />

 <!-- Email input field -->
 <label for="email">Email:</label>
 <input type="email" id="email" ng-model="user.email" required />

 <!-- Password input field -->
 <label for="password">Password:</label>
 <input type="password" id="password" ng-model="user.password" required />

 <!-- Gender radio buttons -->
 <label for="gender">Gender:</label>

 <input type="radio" id="male" ng-model="user.gender" value="Male" /> Male

 <input type="radio" id="female" ng-model="user.gender" value="Female" /> Female

 <!-- Hobbies checkbox -->
 <label for="hobbies">Hobbies:</label>

 <input type="checkbox" id="reading" ng-model="user.hobbies.reading" /> Reading

 <input type="checkbox" id="sports" ng-model="user.hobbies.sports" /> Sports

 <!-- Country select dropdown -->
 <label for="country">Country:</label>
 <select id="country" ng-model="user.country" required>
 <option value="">Select a country:/option>
 <option value="India">India</option>
 <option value="USA">USA</option>
 <option value="UK">UK</option>
 </select>

 <!-- Submit and Reset buttons -->
 <button type="submit">Submit</button>
 <button type="button" ng-click="resetForm()">Reset</button>
 </form>
 <!-- Form ends -->

 <!-- Display the form data below after submission -->
 <div ng-if="submitted">
 <h3>Form Data</h3>
 <p>Name: {{ user.name }}</p>
 <p>Email: {{ user.email }}</p>
 <p>Password: {{ user.password }}</p>
 <p>Gender: {{ user.gender }}</p>
 <p>Hobbies:
 Reading
 Sports
 </p>
 <p>Country: {{ user.country }}</p>
 </div>

</body>
</html>

<script>
 // Define the AngularJS module
 var app = angular.module('myApp', []);

 // Define the AngularJS controller
 app.controller('myCtrl', function($scope) {
 // Initialize user object
 $scope.user = {
 name: '',
 email: '',
 password: '',
 gender: '',
 hobbies: {
 reading: false,
 sports: false
 }
 };
 });
</script>
```

# Modules

An AngularJS **module** is a container for the different parts of an AngularJS application (controllers, services, directives, filters, etc.). It acts as the root of the application and provides a way to organize the code into logical units. Modules help manage dependencies and allow for the reuse of code across different applications.

# Modules

An AngularJS module defines an application.

The module is a container for the different parts of an application.

The module is a container for the application controllers.

Controllers always belong to a module.

A module is created by using the AngularJS function `angular.module`

```
<div ng-app="myApp">...</div>
```

```
<script>
```

```
var app = angular.module("myApp", []);
```

```
</script>
```

# AngularJS Table

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8" >
 <meta name="viewport" content="width=device-width, initial-scale=1.0" >
 <title>AngularJS Table Example </title>
 <!-- Include AngularJS from CDN -->
 <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js" ></script>
</head>
<body>

<div ng-app="myApp" ng-controller="customersCtrl" >

 <table>
 <tr>
 <th>#</th>
 <th>Name</th>
 <th>Country</th>
 </tr>
 <tr ng-repeat="x in names | orderBy:'Country'" >
 <td>{{ $index + 1 }} </td>
 <td>{{ x.Name }} </td>
 <td>{{ x.Country | uppercase }} </td>
 </tr>
 </table>

</div>

<script>
var app = angular.module('myApp', []);
app.controller('customersCtrl', function($scope) {
 // input data
 $scope.names = [
 { Name: "John", Country: "USA" },
 { Name: "Anna", Country: "Germany" },
 { Name: "Shyam", Country: "India" },
 { Name: "Paul", Country: "France" },
 { Name: "Mark", Country: "Canada" },
 { Name: "Sara", Country: "Australia" }
];
});
</script>

</body>
</html>
```

# HTML DOM

The HTML DOM is an Object Model for HTML. It defines:

- HTML elements as objects
- Properties for all HTML elements
- Methods for all HTML elements
- Events for all HTML elements
- JavaScript can add/change/remove HTML events

```
<div ng-app="" ng-init="mySwitch=true">
<p>
<button ng-disabled="mySwitch">Click Me!</button>
</p>
<p>
<input type="checkbox" ng-model="mySwitch">Button
</p>
<p>
{{ mySwitch }}
</p> </div>
```

The ng-show directive shows or hides an HTML element.

```
<div ng-app="">
```

```
<p ng-show="true">I am visible.</p>
```

```
<p ng-show="false">I am not visible.</p>
```

```
</div>
```

The ng-show directive shows (or hides) an HTML element based on the value of ng-show.

```
<div ng-app="" ng-init="hour=13">
<p ng-show="hour > 12">I am visible.</p>
</div>
```

[https://www.w3schools.com/angular/tryit.asp?filename=try\\_ng\\_html\\_show\\_if](https://www.w3schools.com/angular/tryit.asp?filename=try_ng_html_show_if)

# The ng-hide Directive

The ng-hide directive hides or shows an HTML element.

```
<div ng-app="">
```

```
<p ng-hide="true">I am not visible.</p>
```

```
<p ng-hide="false">I am visible.</p>
```

```
</div>
```

# Services

In AngularJS you can make your own service, or use one of the many built-in services.

What is a Service?

In AngularJS, a service is a function, or object, that is available for, and limited to, your AngularJS application.

AngularJS has about 30 built-in services. One of them is the `$location` service.

The `$location` service has methods which return information about the location of the current web page:

# `$location` service

Use the `$location` service in a controller:

```
var app = angular.module('myApp', []);
app.controller('customersCtrl', function($scope, $location)
{
 $scope.myUrl = $location.absUrl();
});
```

## Why use Services?

For many services, like the `$location` service, it seems like you could use objects that are already in the DOM, like the `window.location` object, and you could, but it would have some limitations, at least for your AngularJS application.

AngularJS constantly supervises your application, and for it to handle changes and events properly, AngularJS prefers that you use the `$location` service instead of the `window.location` object.

## The \$http Service

The `$http` service is one of the most common used services in AngularJS applications. The service makes a request to the server, and lets your application handle the response.

Use the `$http` service to request data from the server:

```
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope, $http) {
 $http.get("welcome.htm").then(function (response) {
 $scope.myWelcome = response.data;
 });
});
```

# The \$timeout Service

The `$timeout` service is AngularJS' version of the `window.setTimeout` function.

Example:

Display a new message after two seconds:

```
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope, $timeout) {
 $scope.myHeader = "Hello World!";
 $timeout(function () {
 $scope.myHeader = "How are you today?";
 }, 2000);
});
```

# The \$interval Service

The `$interval` service is AngularJS' version of the `window.setInterval` function.

## Example

Display the time every second:

```
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope, $interval) {
 $scope.theTime = new Date().toLocaleTimeString();
 $interval(function () {
 $scope.theTime = new Date().toLocaleTimeString();
 }, 1000);
});
```

# Create Your Own Service

To create your own service, connect your service to the module:

```
<script>

var app = angular.module('myApp', []);

app.service('square', function() {

 this.myFunc = function (x) {

 return x*x;

 }

});

app.controller('myCtrl', function($scope, square) {

 $scope.s = square.myFunc(5);

}); </script>
```

# Create Your Own Service

```
<div ng-app="myApp" ng-controller="myCtrl">
<p>The square of 5 is:</p>
<h1>{{s}}</h1>
</div>
```

# AJAX

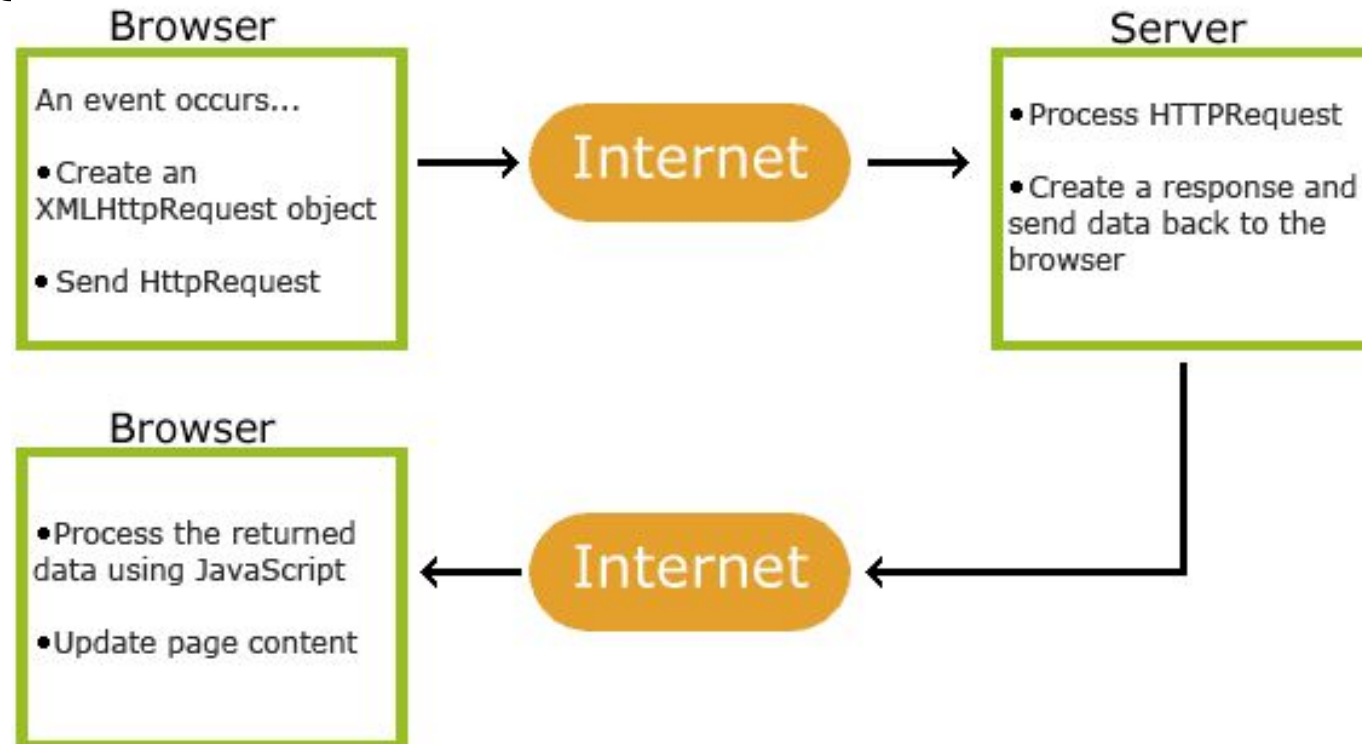
- Update a web page without reloading the page
- Request data from a server - after the page has loaded
- Receive data from a server - after the page has loaded
- Send data to a server - in the background
- AJAX is not a programming language.
- AJAX is a technique for accessing web servers from a web page.
- AJAX stands for Asynchronous JavaScript And XML.

AJAX uses a combination of:

- A browser built-in XMLHttpRequest object (to request data from a web server)
- JavaScript and HTML DOM (to display or use the data)

# Execution flow

AJAX allows web pages to be updated asynchronously by exchanging data with a web server behind the scenes. This means that it is possible to update parts of a web page, without reloading the whole page



# Execution flow

1. An event occurs in a web page (the page is loaded, a button is clicked)
2. An XMLHttpRequest object is created by JavaScript
3. The XMLHttpRequest object sends a request to a web server
4. The server processes the request
5. The server sends a response back to the web page
6. The response is read by JavaScript
7. Proper action (like page update) is performed by JavaScript